

BestSupport Application Workflow Documentation

Scope: application logic only. This document intentionally excludes Docker, deployment, Kubernetes, MinIO, object storage, and infrastructure configuration.

1. Application Purpose

BestSupport is a Django helpdesk application for creating, assigning, approving, discussing, resolving, and closing support work. The main business flows are normal helpdesk tickets, CBS access requests, remote access requests, incident reports, ticket chat, audio-call signaling, technical documents, and portal announcements.

2. Main Application Areas

Area	Main Files	Business Responsibility
Project shell	<code>helpdesk/helpdesk/urls.py</code> , <code>health.py</code> , <code>asgi.py</code>	Root URL routing, health endpoints, and HTTP/WebSocket application entry points.
Accounts	<code>accounts/models.py</code> , <code>forms.py</code> , <code>views.py</code> , <code>backends.py</code>	User profile data, authentication, signup, password reset, branches, departments, and active-session tracking.
Tickets	<code>tickets/models.py</code> , <code>forms.py</code> , <code>views.py</code> , <code>consumers.py</code>	Ticket lifecycle, approvals, incident reports, chat, calls, queues, documents, notifications, and status changes.
Templates	<code>templates/base.html</code> , <code>templates/tickets/*.html</code> , <code>templates/accounts/*.html</code>	User-facing pages rendered by the account and ticket views.
Static UI	<code>static/css/local-ui.css</code> , <code>static/js/local-ui.js</code>	Shared interface styling and small browser-side behavior.

3. Request Routing Flow

1. A browser request reaches Django.
2. `helpdesk/helpdesk/urls.py` routes it to `admin`, `accounts`, `tickets`, `health`, or the root login redirect.
3. Account routes are handled through `accounts.urls`; ticket routes are handled through `tickets.urls`.
4. The matched view validates login state, permissions, form data, and model state.
5. The view reads or updates models, sends notifications or email when needed, and renders a template or redirects.
6. For live chat, audio-call signaling, and browser notifications, WebSocket requests are routed through `tickets/routing.py` to consumers in `tickets/consumers.py`.

4. Key Models

Model	App Logic Role
CustomUser	User account plus support flags, Central Operation flag, branch, department, position, phone, and signature image.
Branch	Named branch records used for user profiles, ticket routing, and document visibility.
Department	Named departments used for ticket routing, support queues, and document visibility.
Ticket	Main work item. Stores requester, assignee, request type, subject, description, branch, department, priority, status, resolution, closure, chat privacy, and display helpers.
RemoteAccessApproval	Approval record for remote access and CBS access requests. Tracks requester, recommender, second recommender, approver, signatures, status, and decision notes.
TicketMessage	Text message in a ticket conversation.
TicketMessageAttachment	File metadata linked to a ticket chat message, including browser-viewable image/PDF helpers.
TicketChatReadState	Tracks when each user last saw chat messages for unread indicators.
TicketAssignmentLog	Assignment and ownership history for reporting and display.
IncidentReport	Structured incident report linked to an incident ticket, including report fields, signers, signatures, submission, correction, and commander data.
IncidentReportSignoff	Ordered signoff rows for notified incident report reviewers.
IncidentReportAttachment	Evidence file metadata linked directly to an incident report.
TechnicalDocument	Knowledge/document entry with branch, department, support-only, or restricted visibility.
PortalFlashAnnouncement	Login/portal announcement shown to users during a configured schedule.
GroupMailboxEmail	Shared mailbox mapping used for department-aware ticket email behavior.

5. Account Workflow

1. A user signs in through `/accounts/login/`.
2. `login_view` authenticates through the configured authentication backend.
3. After login, users are redirected into the ticket/dashboard area.
4. Signup and password reset routes are controlled by authentication feature flags.
5. User department, branch, support role, Central Operation role, and signature image affect permissions throughout ticket, CBS, and incident workflows.

6. Normal Ticket Workflow

1. User opens `/tickets/new/`.
2. `create_ticket` builds `TicketForm` for the current user.
3. The form validates request type, department, branch, notify email, assignment email, impact, urgency, CC emails, and attachments.
4. On valid submit, a `Ticket` is created with a generated ticket ID and calculated priority.
5. If an assignment email is supplied, the matching user becomes the assignee. If only notify email is supplied, that address receives notification but does not own the ticket.
6. Department users can see department-routed tickets and may claim unassigned tickets when allowed.
7. Support users work tickets through support queues and the ticket update page.
8. Resolved tickets record resolution note, resolved user, resolved timestamp, and email the requester with a close link.
9. Closed tickets become read-only for chat and cannot be reopened through the normal ticket panel.

7. Ticket Status Rules

Status Area	Important Rule
Assignment	Unassigned department tickets may be claimed by eligible department users. Created-by users cannot claim their own ticket unless they are also assigned through the workflow.
Resolution	A ticket must be assigned before it can be resolved. Normally, only the assigned agent can mark it resolved.
Closure	A ticket can be closed only after it is resolved. Closing is restricted to an admin user or effective assignee.
Resolved/closed reopening	Resolved and closed tickets cannot be reopened through the normal ticket update panel.
Incident resolution	Incident tickets require required incident report conditions before resolution is accepted.
Approved CBS resolution	When an approved CBS access request is resolved, document sending, ticket update, and new chat messages are locked.

8. Ticket Detail Page

`ticket_detail` is the central page for a ticket. Its context decides which blocks and actions the user can see.

Context Area	What It Controls
Requester info	Shows requester details when the user is allowed to see them.

Context Area	What It Controls
Approval data	Shows remote access or CBS access request details, signatures, status, and reviewer actions.
Incident data	Shows incident report availability, edit/view actions, and download actions.
Chat state	Loads visible chat messages, unread state, lock message, upload limits, and WebSocket settings.
Action flags	Controls buttons such as update ticket, claim ticket, approve/reject, assign approved CBS access, send approved document, and manage chat privacy.
Attachments	Shows ticket-level image attachment, chat attachments, CBS request attachments, and incident evidence links according to permissions.

9. Chat and Call Workflow

1. Ticket detail renders the `Ticket Chat` panel if the current user has chat access.
2. `can_access_ticket_chat` decides chat membership.
3. Normal ticket chat membership is based on requester, assignee, support/admin users, department membership, and private-chat settings.
4. CBS access request chat membership is based on requester, assignee, current approval-chain users, users who already acted, Central Operation, IT support, and admins.
5. Remote access approval requests that are not CBS access requests do not use ticket chat.
6. Chat messages are sent over `TicketChatConsumer`, saved as `TicketMessage`, and broadcast to connected participants.
7. Attachments are uploaded by the ticket attachment upload view and then announced in the chat stream.
8. Audio calls use `TicketCallConsumer` only for signaling. Call permission follows the same chat access helper.
9. Closed tickets disable chat and file upload. Approved CBS access requests also disable chat after the linked ticket is resolved.

10. CBS Access Request Workflow

1. Head Office requests start at `/tickets/cbs-access-request/`.
2. Branch requests start at `/tickets/cbs-access-request/branch/`.
3. Both routes use the CBS access request view logic with office-specific fields.
4. `CBSAccessRequestForm` validates requester details, access user, user groups, endorsement, recommender, second recommender for branch requests, approver, and attachments.
5. The app creates a ticket and linked `RemoteAccessApproval`.
6. The current reviewer can recommend or reject. Branch requests can require a second recommendation before final approval.
7. Central Operation can adjust the signoff chain before final approval when allowed.
8. When final approval is recorded, the approved CBS document is generated from saved request and approval data.

9. The requester can assign the approved request to a concerned user and email the signed document while the ticket is still open.
10. After final approval and ticket resolution, CBS document sending and ticket status editing are locked.

CBS Chat Membership

User Type	Chat Access
Requester	Can participate.
Assigned user	Can participate.
Current recommender, second recommender, approver	Can participate while they are in the current approval chain.
Users who already recommended, second-recommended, or approved	Keep access as historical actors.
Central Operation	Can participate for CBS access tickets.
IT support/admin	Can participate.
Removed reviewer who never acted	Does not keep chat access after being removed from the chain.

11. Remote Access Request Workflow

1. User opens `/tickets/remote-access-request/`.
2. `RemoteAccessRequestForm` validates subject, details, recommender, and approver.
3. The app creates a ticket and a linked `RemoteAccessApproval`.
4. The approval record tracks pending recommendation, pending approval, approved, or rejected state.
5. Reviewer decisions are handled by `remote_access_approval_update`.
6. Remote access approval requests do not use the normal ticket chat flow.

12. Incident Report Workflow

1. An incident ticket can have a linked `IncidentReport`.
2. `ticket_incident_report` renders and processes the report form.
3. Support users, the requester, and the incident commander can manage the report when allowed.
4. The report tracks incident information, containment, recovery, lessons learned, action plan, notification data, and evidence notes.
5. Evidence files are listed with view/download actions when the current user can access the incident report.
6. Registered user and notified signoff users sign in sequence according to the saved signoff rows.
7. Incident reports can be saved as draft, submitted, sent to reviewers/approvers, corrected, or locked depending on state.
8. Incident ticket resolution is blocked until required report and signature conditions are complete.

Incident Report Access

User Type	Access
Ticket participant	Can view the incident report.
Incident commander	Can view and manage where allowed.
Registered user	Can view/sign according to role.
Notified user or signoff user	Can view/sign according to sequence.
Support/admin	Can view and manage according to support permissions.

13. Technical Documents Workflow

1. Users open `/tickets/docs/` to see documents visible to them.
2. Support users can upload documents through `tech_docs_upload`.
3. Visibility can be public, branch-based, department-based, restricted to selected users, or support-only.
4. View and download actions check document visibility before serving content.
5. Supported spreadsheet documents can be previewed in a browser-friendly table view.

14. Portal Flash Announcements

1. Support/admin users upload announcements through the portal flash upload page.
2. Each announcement has title, message, image, visibility, and schedule fields.
3. Active announcements are shown to users after login according to schedule and visibility rules.

15. Support Queues

Queue	Purpose
<code>support_dashboard</code>	High-level support dashboard with counts, workload, and recent tickets.
<code>support_queue</code>	Main ticket queue with filters for ticket ID, status, priority, requester, assignee, branch, department, and dates.
<code>support_department_tickets</code>	Department-focused view split into unassigned, assigned, and solved groups.
<code>support_cbs_access_requests</code>	CBS-specific request queue for support/Central Operation users.
<code>support_incident_tickets</code>	Incident-specific support queue.
<code>support_users</code>	Currently logged-in users and related support visibility.

16. Important URL Groups

URL Pattern	Business Use
<code>/tickets/</code>	My Tickets list.
<code>/tickets/new/</code>	Create normal ticket.
<code>/tickets/<ticket_id>/</code>	Ticket detail page.
<code>/tickets/support/<ticket_id>/update/</code>	Ticket assignment/status update.
<code>/tickets/<ticket_id>/claim/</code>	Claim eligible department ticket.
<code>/tickets/<ticket_id>/attachments/upload/</code>	Add file to ticket chat.
<code>/tickets/<ticket_id>/attachments/<attachment_id>/view/</code>	View supported ticket attachment inline.
<code>/tickets/<ticket_id>/attachments/<attachment_id>/download/</code>	Download ticket attachment.
<code>/tickets/cbs-access-request/</code>	Create Head Office CBS access request.
<code>/tickets/cbs-access-request/branch/</code>	Create Branch CBS access request.
<code>/tickets/<ticket_id>/remote-access-approval/</code>	Recommend, approve, or reject remote/CBS request.
<code>/tickets/<ticket_id>/cbs-access-request/send-document/</code>	Email approved CBS document while allowed.
<code>/tickets/<ticket_id>/incident-report/</code>	Create, view, edit, submit, or sign incident report.
<code>/tickets/docs/</code>	Technical document list.

17. Developer Takeover Map

A new developer should treat `tickets/views.py` as the main business-orchestration file and `tickets/models.py` as the source of workflow state and permission helper behavior. Templates decide what the user sees, but important permissions must also be enforced in views and consumers.

Need To Change	Start Here	Then Check
Ticket creation fields or validation	<code>tickets/forms.py</code> → <code>TicketForm</code>	<code>tickets/views.py</code> → <code>create_ticket</code> , then <code>templates/tickets/create_ticket.html</code>
Ticket detail actions or visibility	<code>tickets/views.py</code> → <code>_build_ticket_detail_context</code>	<code>templates/tickets/ticket_detail.html</code>
Support status update rules	<code>tickets/views.py</code> → <code>_ticket_update_protected</code>	<code>tickets/forms.py</code> → <code>TicketUpdateForm</code> , <code>TicketAssigneeUpdateForm</code>

Need To Change	Start Here	Then Check
Chat access or notification participants	<code>tickets/models.py</code> → <code>can_access_ticket_chat</code> , <code>get_ticket_chat_access_user_ids</code>	<code>tickets/consumers.py</code> , <code>tickets/notifications.py</code> , <code>tickets/chat_rules.py</code>
CBS access request fields	<code>tickets/forms.py</code> → <code>CBSAccessRequestForm</code>	<code>tickets/views.py</code> CBS helper functions and <code>templates/tickets/cbs_access_request.html</code>
CBS approval-chain behavior	<code>tickets/views.py</code> → <code>remote_access_approval_update</code> , <code>cbs_access_signoff_chain_update</code>	<code>RemoteAccessApproval</code> in <code>tickets/models.py</code>
Incident report fields or signoff behavior	<code>tickets/forms.py</code> → <code>IncidentReportForm</code>	<code>tickets/views.py</code> incident helper functions and <code>templates/tickets/incident_report.html</code>
Document visibility rules	<code>tickets/views.py</code> → <code>_can_view_tech_doc</code> , <code>_tech_doc_visibility_q</code>	<code>TechnicalDocument</code> in <code>tickets/models.py</code>
User role/profile behavior	<code>accounts/models.py</code> → <code>CustomUser</code> , <code>Department</code> , <code>Branch</code>	<code>accounts/views.py</code> , <code>accounts/admin.py</code> , and relevant ticket permission helpers

18. Permission Helpers and Access Rules

The project relies on small helper functions instead of scattering permission logic everywhere. When changing access behavior, update the helper and then confirm every direct URL path still enforces it.

Helper	Location	Used For
<code>_is_support_user</code>	<code>tickets/views.py</code>	Identifies staff, superusers, and IT support users for support queues/actions.
<code>_is_central_operation_user</code>	<code>tickets/views.py</code>	Identifies Central Operation users for CBS-specific visibility and actions.
<code>_is_ticket_participant</code>	<code>tickets/views.py</code>	Controls ticket detail and attachment access for requester, assignee, support, department members, CBS reviewers, and incident actors.
<code>can_access_ticket_chat</code>	<code>tickets/models.py</code>	Controls chat, call signaling, chat message visibility, upload, delete, seen state, and WebSocket connection.
<code>get_ticket_chat_access_user_ids</code>	<code>tickets/models.py</code>	Builds chat and call notification target users.
<code>can_manage_ticket_chat_privacy</code>	<code>tickets/models.py</code>	Controls whether a ticket's private-chat setting can be changed. Approval tickets do not expose this

Helper	Location	Used For
		setting.
<code>_can_access_incident_report</code>	<code>tickets/ views.py</code>	Controls incident report pages, evidence attachments, signatures, and report downloads.
<code>_can_manage_incident_report</code>	<code>tickets/ views.py</code>	Controls who can create/update incident report content.
<code>_can_decide_remote_access_approval</code>	<code>tickets/ views.py</code>	Controls recommend/approve/reject buttons and POST behavior for remote/CBS approvals.
<code>_can_manage_cbs_access_signoff_chain</code>	<code>tickets/ views.py</code>	Controls Central Operation signoff-chain changes before final CBS approval.

Permission Change Rule

1. Never rely only on hiding a button in a template.
2. Enforce the same rule in the view handling the direct URL or POST.
3. If WebSockets are involved, enforce the rule in the consumer access check too.
4. If files are involved, enforce the rule on both view and download URLs.
5. Add tests for both successful access and blocked access.

19. State Machines and Business Status

Ticket Status

Status	Meaning	Important Behavior
<code>new</code>	Ticket has been created and is waiting for action.	Visible in queues according to owner/department/role.
<code>acknowledged</code>	Someone has seen/accepted the work but it is not yet in progress.	Still active.
<code>in_progress</code>	Work is actively being handled.	Active support workflow.
<code>waiting_on_user</code>	Support is waiting for requester/user input.	Active but blocked externally.
<code>waiting_on_third_party</code>	Support is waiting for outside/vendor/team input.	Active but blocked externally.
<code>resolved</code>	Support considers the issue fixed.	Requester receives email with close link. CBS approved requests become locked for document sending and status editing.
<code>closed</code>	Ticket is complete.	Chat and uploads are disabled. Normal panel

Status	Meaning	Important Behavior
		cannot reopen.
cancelled_duplicate	Ticket was cancelled as duplicate/not needed.	Treated as a solved state in queues.

Remote/CBS Approval Status

Status	Meaning	Typical Next Step
pending_recommendation	Request is waiting for first or second recommendation.	Current recommender approves or rejects.
pending_approval	Required recommendation stage is done; final approver must decide.	Approver approves or rejects.
approved	Final approval is complete.	CBS request can be assigned/processed while ticket remains open.
rejected	Reviewer rejected the request.	Requester can correct/resubmit if workflow allows.

Incident Report State

State	How It Is Represented	Effect
Draft	Report exists but is not submitted.	Allowed managers can edit report fields and evidence.
Waiting for signatures	Required signature fields or signoff rows are incomplete.	Submit/resolution may be blocked until signatures are complete.
Submitted	submitted_at and submitted_by are set.	Report is treated as formally sent/submitted.
Correction requested	Correction fields are set.	Report can be returned for changes depending on role and lock rules.
Finalized/locked	Workflow lock helps determine finality.	Editing is blocked through normal views.

20. Detailed Workflow: Normal Ticket

1. User opens the create ticket page.
2. The form preloads user context such as available departments, branches, and support routing options.
3. User selects request type, department/branch, impact, urgency, notify email, assignment email, CC emails, description, and optional attachments.
4. The form validates email behavior. Assignment email must map to a real user and must not be a group mailbox. Notify email can be individual or shared mailbox according to rules.

5. Priority is calculated from impact and urgency.
6. The ticket is created with requester set to the current user.
7. Assignment is applied if requested. Otherwise, department routing and support queues determine visibility.
8. Notification email is sent to configured recipients.
9. Requester lands on the ticket detail or list page and can continue in chat.
10. Support users pick up work from support queues, claim if eligible, update status, and resolve/close according to rules.

21. Detailed Workflow: CBS Access Request

1. Requester opens either Head Office or Branch CBS request form.
2. The form captures user details, requested CBS user groups, request type, amendment reason where applicable, access user, signoff users, and supporting attachments.
3. On submit, the app creates a ticket with request type `cbs_access` or `cbs_access_branch`.
4. The app creates a linked `RemoteAccessApproval` with recommender, optional second recommender, and approver.
5. The request detail appears inside the ticket detail page using saved ticket and approval data.
6. The current reviewer receives the request and can recommend/reject.
7. For branch requests, a second recommendation can be required before final approval.
8. The final approver approves or rejects.
9. Approved request displays signed approval information and allows allowed users to send/assign the approved document while the ticket remains active.
10. CBS request chat is available to current chain users, historical signers, requester, assignee, Central Operation, support, and admins.
11. If Central Operation changes the chain before a reviewer acts, the old reviewer loses chat access. If the old reviewer already acted, they keep access as a historical signer.
12. Once approved and the linked ticket is resolved, approved document sending, ticket update, and new chat messages are locked.

22. Detailed Workflow: Incident Report

1. Incident ticket is created or selected.
2. Allowed user opens the incident report page from ticket detail.
3. The report form captures incident timeline, service impact, branch impact, containment, recovery, root cause, lessons learned, recommendations, notification details, and action plan.
4. Evidence files can be uploaded and later viewed/downloaded by users who can access the incident report.
5. Registered user signature is copied from the user's profile signature when available.
6. Notified signoff rows define ordered reviewers/signers.
7. Signoff users can sign or reject according to sequence and role.
8. Submit sends the report to configured recipients when required fields/signatures are complete.
9. Incident ticket resolution checks report completion rules first.
10. After finalization, normal editing is blocked.

23. File View and Download Logic

This section documents only application permission behavior for files. It does not describe file backend implementation or infrastructure.

File Type	Where User Sees It	Access Rule
Ticket image	Ticket detail attachment block	Visible to ticket participants.
Ticket message attachment	Ticket chat and CBS attachment list	Visible to chat participants. CBS access request participants can view request attachments even if they are not normal chat users.
Incident evidence attachment	Incident report page	Visible to users who can access the incident report.
Technical document	Technical documents page	Visible according to document visibility: public, branch, department, restricted, or support-only.

24. Email and Notification Logic

Trigger	What Happens
Ticket created	Requester/support recipients receive new ticket notification based on assignment and notify-email rules.
Ticket assigned	Assigned user is notified and may receive assignment email.
Ticket resolved	Requester receives resolution email with close link, resolution details, optional CC, and optional resolved-email attachments.
Ticket chat message	Eligible chat participants except the sender receive browser notification payloads.
Ticket audio call	Eligible chat/call participants receive incoming call notification payloads.
Remote/CBS approval stage changes	Current reviewer or approver receives email/notification where configured.
Approved CBS request assigned	Concerned user receives assignment email with signed document attached.
Incident report submit/signoff	Relevant reviewers, approvers, signers, and CC recipients receive report-related email/notification.

25. Template Ownership

Template	Purpose	Key Context Variables
<code>templates/base.html</code>	Global layout, sidebar,	User, auth flags, active navigation, flash

Template	Purpose	Key Context Variables
	header, notifications, and shared page frame.	announcements.
<code>templates/tickets/ticket_list.html</code>	User ticket list and filters.	<code>tickets</code> , selected filters, unread flags, claim/update flags.
<code>templates/tickets/create_ticket.html</code>	Normal ticket create form.	<code>form</code> , field choices, validation errors.
<code>templates/tickets/ticket_detail.html</code>	Main ticket detail, CBS details, incident link, chat, attachments, assignment history, and actions.	<code>ticket</code> , <code>remote_access_approval</code> , <code>can_update_ticket</code> , <code>can_view_chat</code> , <code>chat_messages</code> , <code>ticket_attachments</code> , CBS/incident flags.
<code>templates/tickets/ticket_update.html</code>	Support/assignee update form.	<code>ticket</code> , <code>form</code> .
<code>templates/tickets/cbs_access_request.html</code>	Head Office and Branch CBS access request forms.	<code>form</code> , office type flags, signoff fields.
<code>templates/tickets/incident_report.html</code>	Incident report form, evidence list, signoff controls, submit/download actions.	<code>form</code> , <code>incident_report</code> , <code>incident_attachments</code> , signoff flags, lock flags.
<code>templates/tickets/support_queue.html</code>	Main support ticket queue.	Filtered tickets and selected filter values.
<code>templates/docs/tech_docs.html</code>	Technical document listing.	<code>documents</code> , visibility and action flags.

26. Testing Guidance for New Developers

The most important tests live in `tickets/tests.py` and `accounts/tests.py`. When changing workflow behavior, add focused tests near the related test class.

Change Type	Test What
Permission change	Allowed user succeeds, blocked user fails, direct URL/POST path enforces the rule, and template does not show forbidden action.
Status transition change	Valid transition succeeds, invalid transition is blocked, saved fields are correct, and user-facing message is shown.
CBS approval change	Each approval stage, final approval/rejection, signoff-chain changes, requester correction, and resolved-lock behavior.
Incident report change	Draft save, submit, signatures, signoff order, evidence access, resolution blockers, and finalized lock behavior.

Change Type	Test What
Chat change	HTTP detail visibility, WebSocket access helper impact, attachment upload, delete, read-state, notification targets, and closed/read-only behavior.
Template action change	Button/link appears for allowed user and is absent for disallowed user; direct backend route must also be tested.

27. Safe Development Checklist

1. Identify the URL and view first.
2. Find the form and model involved.
3. Find existing permission helpers before adding new checks.
4. Update both UI visibility and server-side enforcement.
5. For chat/call changes, confirm WebSocket consumer permissions still match HTTP permissions.
6. For attachments/documents, confirm view and download URLs use the same intended access rule.
7. For approval or incident changes, test both the current actor and a user who should not be allowed.
8. Keep status-locking rules explicit; avoid changing resolved/closed behavior accidentally.
9. Add or update focused tests before broader refactors.
10. Do not mix unrelated UI, workflow, and data-model changes in one patch unless the feature truly needs all three.

28. How to Understand or Extend a Feature

1. Start with `helpdesk/helpdesk/urls.py` to find the application route.
2. Open the matching app `urls.py` to identify the view function.
3. Read the view in `views.py` for permission checks, model queries, form validation, state changes, notifications, email, and template context.
4. Read the form in `forms.py` for user-facing fields and validation rules.
5. Read the model in `models.py` for stored fields, helper properties, status choices, and relationships.
6. Open the rendered template to understand which actions are visible to each user.
7. For real-time behavior, read `asgi.py`, `tickets/routing.py`, and `tickets/consumers.py`.
8. For permission-sensitive changes, add tests around both the UI visibility and the direct URL/POST path.

■ This document describes only application-level workflow and business logic. It does not document
 ■ Docker, deployment, Kubernetes, MinIO, object storage, or server infrastructure.